

Platforma de predicții de business folosind Machine Learning

Estimarea prețului locuințelor și predicția pierderii clienților

Proiect Machine Learning - 2025-2026

Aplicația demonstrează un pipeline complet de Machine Learning pentru două tipuri de probleme cerute în proiect: regresie și clasificare. Modulul de regresie estimează prețul unei case din King County, iar modulul de clasificare estimează riscul ca un client telecom să renunțe la serviciu.



Figura 1. Fluxul general al proiectului, de la dataseturile Kaggle până la GUI-ul Streamlit.

1. Scopul aplicației

Scopul proiectului este să arate cum pot fi folosite metodele studiate la materia Machine Learning într-un produs minimal, dar complet. Aplicația nu încearcă să lege direct cele două dataseturi, deoarece ele provin din domenii diferite. Legătura dintre ele este metodologică: același mod de lucru este aplicat unei probleme de regresie și unei probleme de clasificare.

- Pentru regresie, intrarea este un set de caracteristici ale unei locuințe, iar ieșirea este un preț estimat.
- Pentru clasificare, intrarea este profilul unui client telecom, iar ieșirea este clasa prezisă: clientul pleacă sau nu pleacă.
- Pentru ambele module, codul face analiză statistică, preprocesare, antrenare, evaluare comparativă și salvare de model.
- GUI-ul Streamlit folosește modelele salvate și permite introducerea unor exemple noi prin formulare.

Cerințe acoperite

- Două dataseturi: unul pentru regresie și unul pentru clasificare.
- Sumare statistice și vizualizări pentru fiecare dataset.
- Transformări de preprocesare prezentate și aplicate în pipeline.
- Câte 5 modele de bază pentru fiecare problemă.
- Câte 2 metode ensemble pentru fiecare problemă.
- Salvarea celui mai bun model și deploy minimal prin GUI.

2. Structura implementării

Proiectul este separat pe responsabilități. Această structură face codul mai ușor de explicat și de modificat: partea de încărcare a datelor este separată de modelare, raportare și interfață.

Fișier	Rol
src/data_preparation.py	Încarcă dataseturile, curăță coloanele, convertește targetul și separă X/y.
src/modeling.py	Definește modelele, preprocesarea, cross-validation, metricile și salvarea artefactelor.
src/reporting.py	Generează sumare statistice, tabele CSV, grafice EDA și sumarul proiectului.
src/train.py	Entry point: rulează pipeline-ul complet pentru ambele dataseturi.
app.py	Interfața Streamlit cu două taburi pentru predicții noi.

3. Datasetul de regresie: House Sales in King County

Datasetul conține vânzări reale de case din King County, Washington. Targetul este coloana `price`, adică prețul de vânzare al locuinței. După curățare au rămas 21.611 observații și 22 de coloane, deoarece data vânzării a fost transformată în componente utile: an, lună și zi din săptămână.

Sumar statistic relevant

Feature	Count	Mean	Std	Min	Median	Max
price	21,611	540,102	367,139	75,000	450,000	7,700,000
bedrooms	21,611	3.3709	0.9301	0.0000	3.0000	33.0000
bathrooms	21,611	2.1148	0.7702	0.0000	2.2500	8.0000
sqft_living	21,611	2,080	918.4464	290.0000	1,910	13,540
grade	21,611	7.6567	1.1754	1.0000	7.0000	13.0000
yr_built	21,611	1,971	29.3722	1,900	1,975	2,015

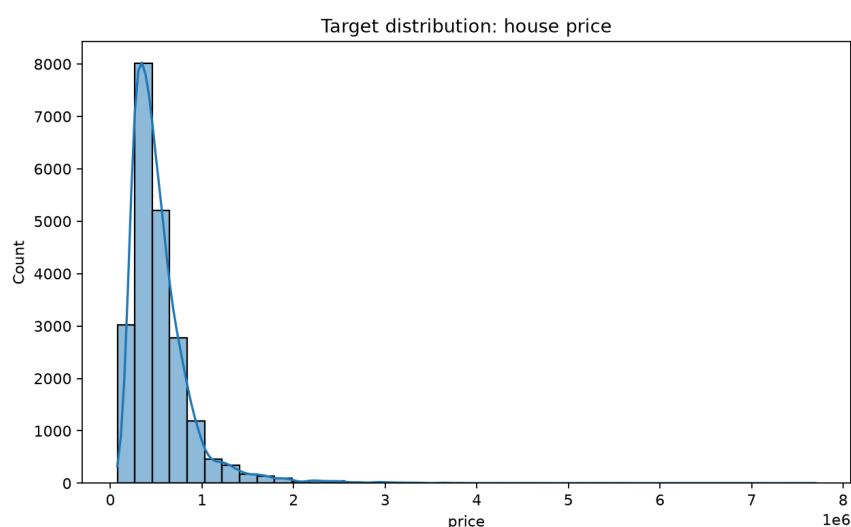


Figura 2. Distribuția targetului `price`. Se observă valori extreme, lucru normal pentru prețuri imobiliare.



Figura 3. Relația dintre suprafața locuibilă și preț. Relația este pozitivă, dar nu perfect liniară.

Corelații și interpretare pentru regresie

Corelația este utilă pentru a observa rapid ce variabile au legătură liniară cu prețul. Cele mai importante corelații apar de obicei pentru suprafață, gradul construcției, calitatea zonei și numărul de băi.

Feature	Corelație cu price
nan	price
sqft_living	0.7020330070628255
grade	0.6675254642802716
sqft_above	0.6055555499881382
sqft_living15	0.5853694226412659
bathrooms	0.5251402077369342
view	0.39728936235929263
sqft_basement	0.32382755685747316
bedrooms	0.30831781399734676
lat	0.3069986869029696

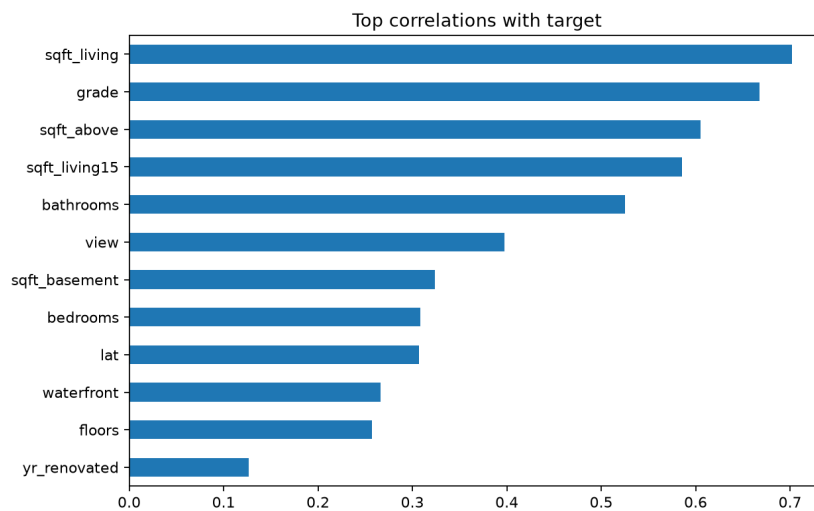


Figura 4. Cele mai mari corelații absolute cu prețul casei.

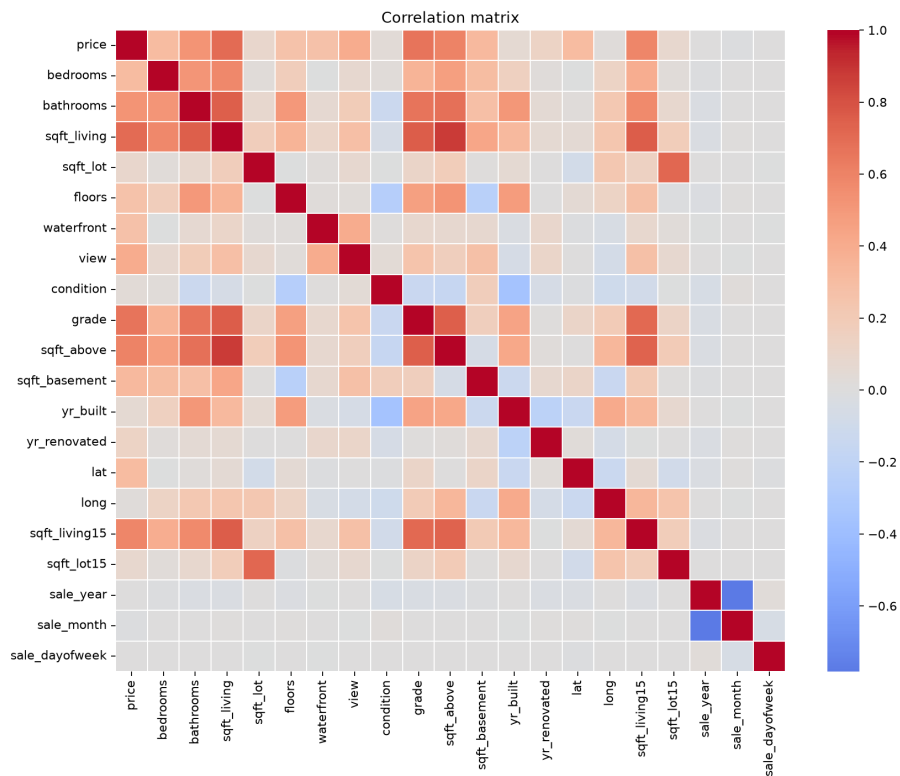


Figura 5. Matricea de corelații pentru variabilele numerice din datasetul de case.

4. Datasetul de clasificare: Telco Customer Churn

Datasetul Telco descrie clienții ai unei companii de telecomunicații. Targetul este `Churn`, convertit în 0 pentru `No` și 1 pentru `Yes`. Problema este de clasificare binară și este ușor dezechilibrată: clienții care nu pleacă sunt mai numeroși decât cei care pleacă.

Sumar statistic relevant

Feature	Count	Mean	Std	Min	Median	Max
tenure	7,021	32.4694	24.5350	0.0000	29.0000	72.0000
MonthlyCharges	7,021	64.8519	30.0690	18.2500	70.4000	118.7500
TotalCharges	7,010	2,290	2,267	18.8000	1,404	8,685
Churn	7,021	0.2645	0.4411	0.0000	0.0000	1.0000

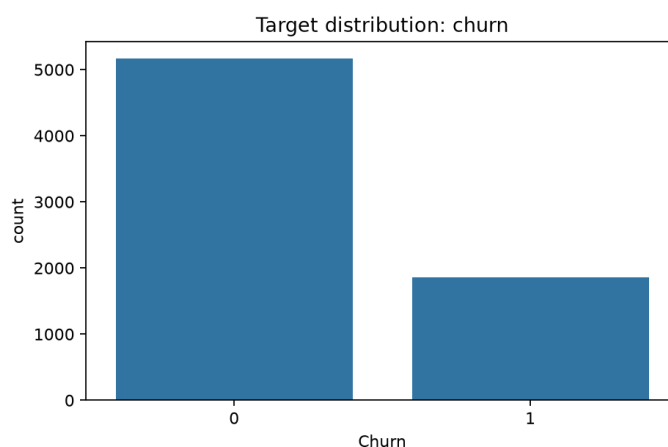


Figura 6. Distribuția claselor pentru `Churn`.

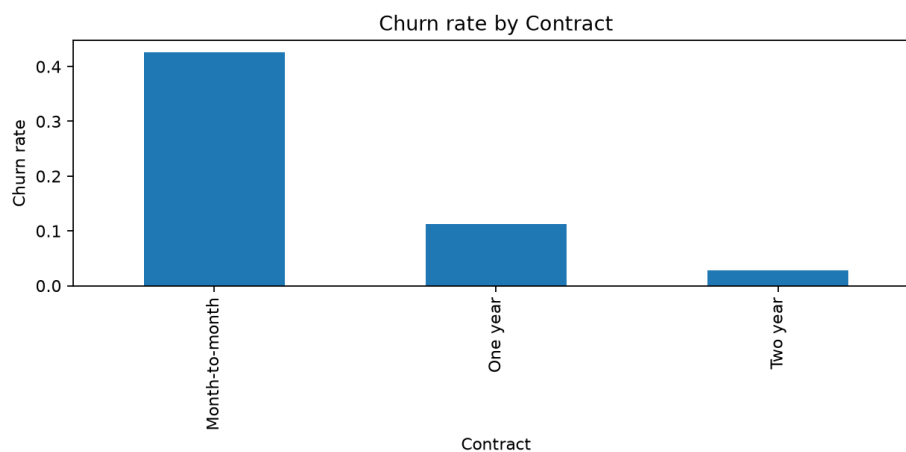


Figura 7. Rata churn în funcție de tipul contractului.

Vizualizări suplimentare pentru churn

Vizualizările arată că variabilele contractuale și comportamentul de facturare sunt foarte relevante. Clienții cu contracte mai flexibile sau metode de plată mai puțin stabile tind să aibă risc mai mare de churn.

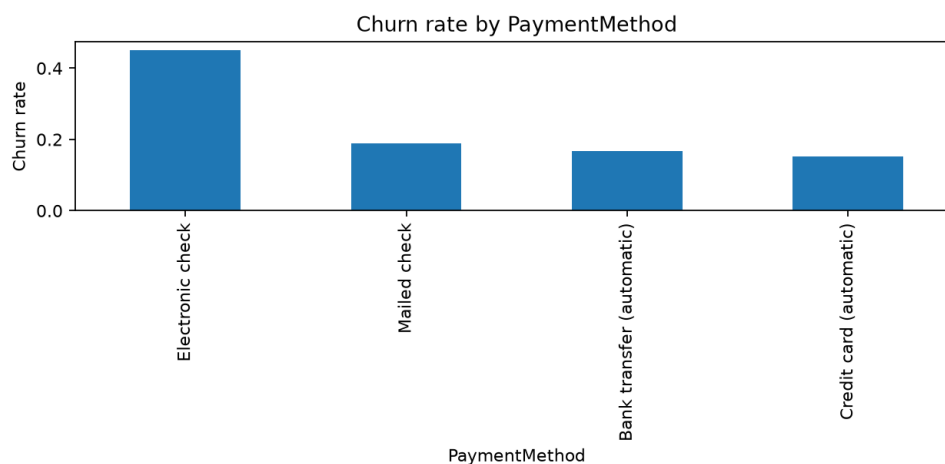


Figura 8. Rata churn în funcție de metoda de plată.

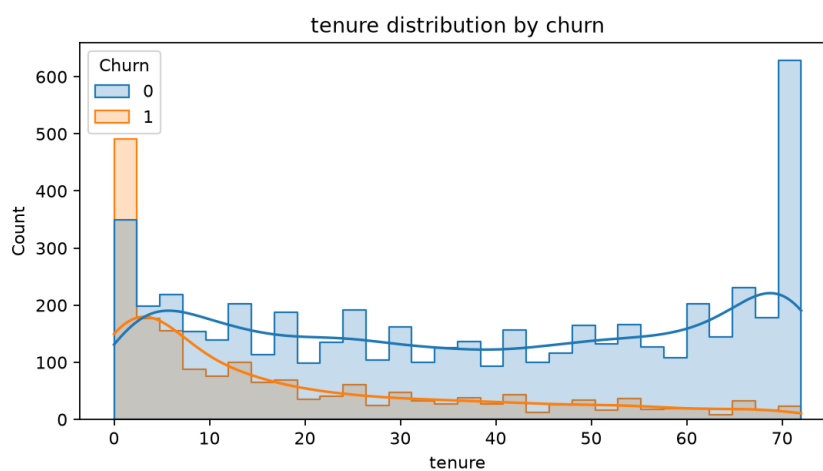


Figura 9. Distribuția vechimii clientului în funcție de churn.

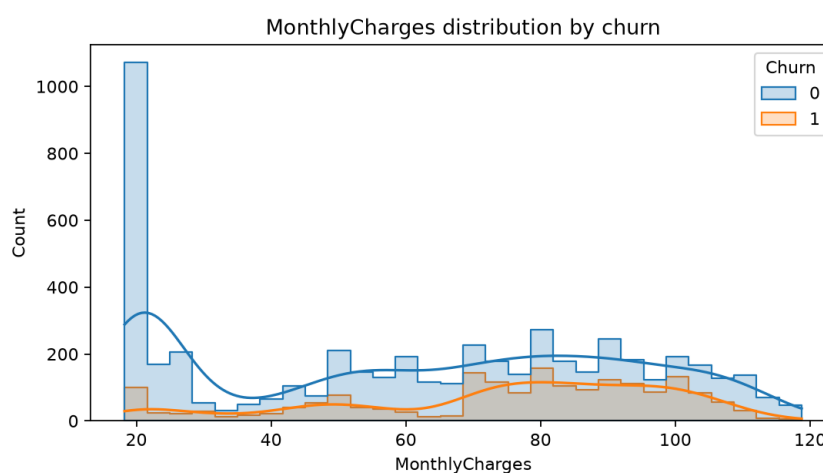


Figura 10. Distribuția costului lunar în funcție de churn.

5. Preprocesarea datelor

Preprocesarea este implementată ca parte din pipeline-ul scikit-learn. Acest lucru este important deoarece aceleași transformări aplicate pe datele de training sunt aplicate automat și pe datele noi introduse în GUI.

- Identificatorii fără valoare predictivă (`id`, `customerID`) sunt eliminați.
- Coloana `date` din datasetul de case este transformată în `sale\_year`, `sale\_month` și `sale\_dayofweek`.
- `TotalCharges` este convertită din text în numeric, iar eticheta `Churn` este mapată la 0/1.
- Valorile lipsă numerice sunt imputate cu mediana; valorile categorice cu moda.
- Variabilele numerice sunt standardizate cu `StandardScaler`.
- Variabilele categorice sunt transformate prin `OneHotEncoder(handle\_unknown='ignore')`.

Fragment de cod - preprocesare

```
def build_preprocessor(X: pd.DataFrame) -> ColumnTransformer:
    numeric_features = [col for col in X.columns if is_numeric_dtype(X[col])]
    categorical_features = [col for col in X.columns if col not in numeric_features]

    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("encoder", _one_hot_encoder()),
        ]
    )

    return ColumnTransformer(
        transformers=[
            ("numeric", numeric_transformer, numeric_features),
            ("categorical", categorical_transformer, categorical_features),
        ],
        remainder="drop",
    )
```

6. Modele antrenate

Pentru fiecare dataset au fost antrenate cinci modele de bază și două modele ensemble. Modelele au fost evaluate prin cross-validation pe setul de training și apoi pe un holdout test set.

Modele pentru regresie

- Linear Regression, Ridge Regression, Lasso Regression.
- KNN Regressor și Decision Tree Regressor.
- Ensemble: Random Forest Regressor și AdaBoost Regressor.

Modele pentru clasificare

- Logistic Regression, KNN Classifier, Gaussian Naive Bayes.
- Decision Tree Classifier și Support Vector Machine.
- Ensemble: Random Forest Classifier și AdaBoost Classifier.

Fragment de cod - lista modelelor

```
def regression_models() -> list[tuple[str, str, Any]]:
    return [
        ("base", "Linear Regression", LinearRegression()),
        ("base", "Ridge Regression", Ridge()),
        ("base", "Lasso Regression", Lasso(max_iter=10000, random_state=RANDOM_STATE)),
    ]
```



```

("base", "KNN Regressor", KNeighborsRegressor(n_neighbors=7)),
(
    "base",
    "Decision Tree Regressor",
    DecisionTreeRegressor(random_state=RANDOM_STATE),
),
(
    "ensemble",
    "Random Forest Regressor",
    RandomForestRegressor(
        n_estimators=200,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    ),
),
(
    "ensemble",
    "AdaBoost Regressor",
    AdaBoostRegressor(n_estimators=100, random_state=RANDOM_STATE),
),
]

```

```

def classification_models() -> list[tuple[str, str, Any]]:
    return [
        (
            "base",
            "Logistic Regression",
            LogisticRegression(max_iter=3000, class_weight="balanced"),
        ),
        ("base", "KNN Classifier", KNeighborsClassifier(n_neighbors=7)),
        ("base", "Gaussian Naive Bayes", GaussianNB()),
        (
            "base",
            "Decision Tree Classifier",
            DecisionTreeClassifier(random_state=RANDOM_STATE, class_weight="balanced"),
        ),
        (
            "base",
            "Support Vector Machine",
            SVC(class_weight="balanced", probability=True, random_state=RANDOM_STATE),
        ),
        (
            "ensemble",
            "Random Forest Classifier",
            RandomForestClassifier(
                n_estimators=200,
                random_state=RANDOM_STATE,
                class_weight="balanced",
                n_jobs=-1,
            ),
        ),
        (
            "ensemble",
            "AdaBoost Classifier",
            AdaBoostClassifier(n_estimators=100, random_state=RANDOM_STATE),
        ),
    ]

```

7. Comparația modelelor de regresie

Pentru regresie au fost urmărite MAE, RMSE și R2. RMSE penalizează mai puternic erorile mari, de aceea este o metrică bună pentru prețuri imobiliare, unde predicțiile foarte greșite sunt costisitoare.

Tip	Model	MAE	RMSE	R2
ensemble	Random Forest Regressor	70,678	138,638	0.8702
base	Ridge Regression	97,043	169,117	0.8069
base	Lasso Regression	97,058	169,125	0.8069
base	Linear Regression	97,062	169,130	0.8069
base	KNN Regressor	92,034	180,962	0.7789
base	Decision Tree Regressor	102,034	202,642	0.7227
ensemble	AdaBoost Regressor	391,833	430,452	-0.2511

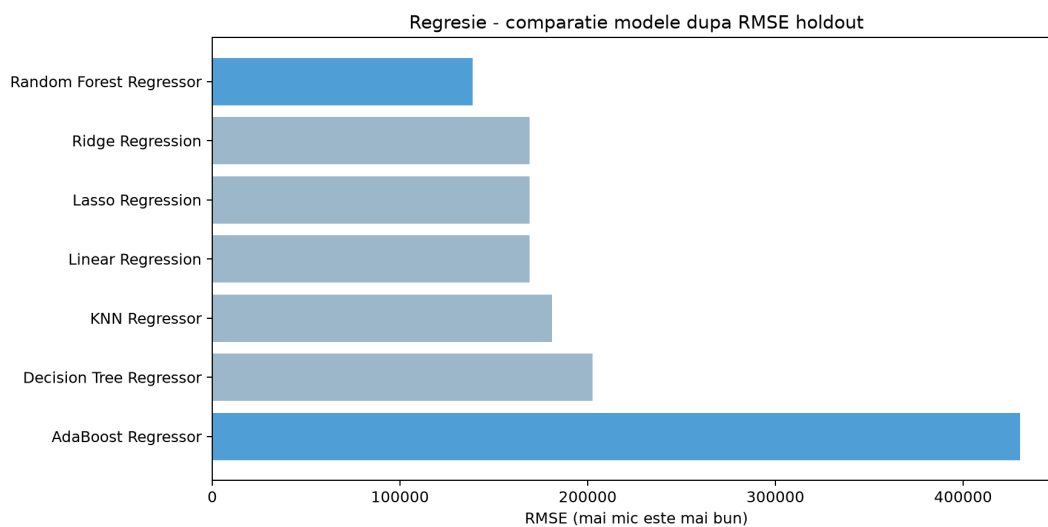


Figura 11. Comparația modelelor de regresie după RMSE pe holdout set.

Modelul selectat este Random Forest Regressor. Acesta obține cel mai mic RMSE și cel mai mare R2, ceea ce indică faptul că surprinde mai bine relațiile neliniare dintre caracteristicile caselor și preț.

8. Comparația modelelor de clasificare

Pentru clasificare au fost urmărite accuracy, precision, recall, F1 și ROC AUC. Într-o problemă de churn, F1 este foarte util deoarece combină precision și recall. Accuracy singură poate fi înșelătoare atunci când clasele nu sunt perfect echilibrate.

Tip	Model	Acc	Prec	Recall	F1	AUC
base	Support Vector Machine	0.7594	0.5315	0.7715	0.6294	0.8245
base	Logistic Regression	0.7402	0.5061	0.7769	0.6129	0.8398
base	Gaussian Naive Bayes	0.6847	0.4488	0.8360	0.5840	0.8067
ensemble	Random Forest Classifier	0.7651	0.5520	0.5995	0.5747	0.8141
ensemble	AdaBoost Classifier	0.7972	0.6537	0.4973	0.5649	0.8424
base	Decision Tree Classifier	0.7480	0.5242	0.5242	0.5242	0.6767
base	KNN Classifier	0.7601	0.5522	0.4973	0.5233	0.7905

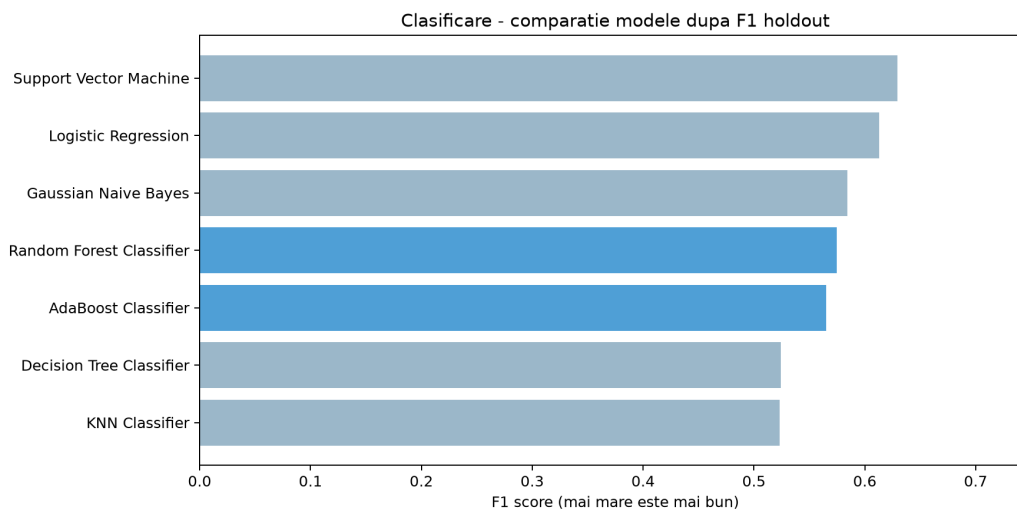


Figura 12. Comparația modelelor de clasificare după F1 pe holdout set.

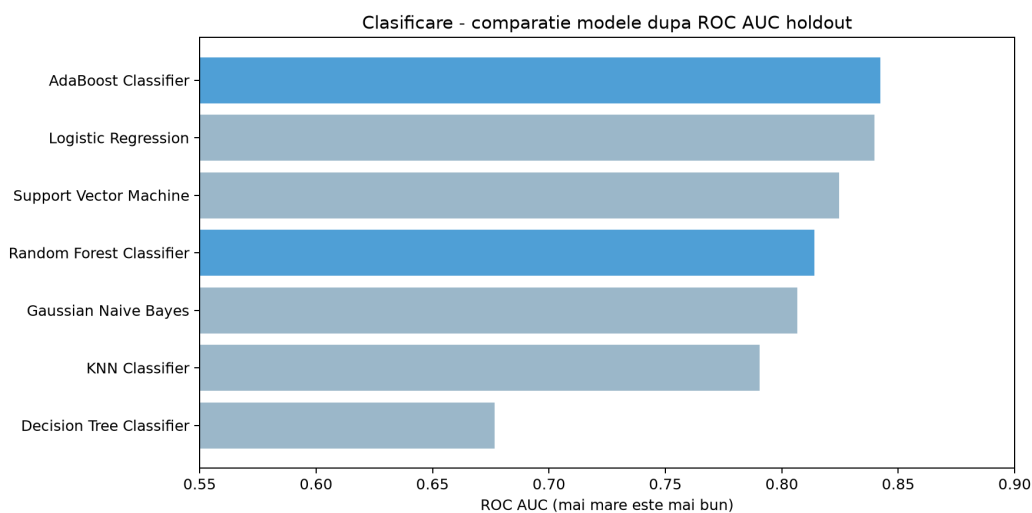


Figura 13. Comparația modelelor de clasificare după ROC AUC pe holdout set.

Interpretarea modelului de clasificare selectat

Modelul selectat după F1 este Support Vector Machine. Are recall ridicat pentru clasa `Yes`, ceea ce înseamnă că identifică o mare parte dintre clienții care chiar pleacă. În aplicații de retenție, acest comportament este util: este de preferat să detectăm cât mai mulți clienți cu risc, chiar dacă apar și câteva alarme false.

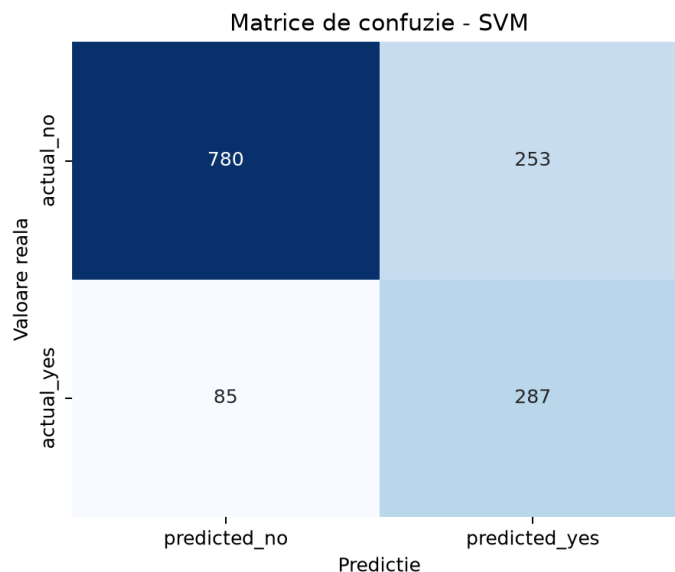


Figura 14. Matricea de confuzie pentru modelul SVM pe holdout set.

Matricea arată 287 clienți churn detectați corect și 85 clienți churn ratați. Există 253 false pozitive, adică persoane marcate cu risc de churn deși nu au plecat. Acest compromis este acceptabil dacă scopul business este prevenirea pierderii clienților, nu doar maximizarea acurateții globale.

9. Ensemble methods

Ensemble methods combină mai multe modele pentru a obține predicții mai stabile. În proiect au fost folosite Random Forest și AdaBoost pentru ambele tipuri de probleme.

- Random Forest construiește mai mulți arbori pe subsample-uri ale datelor și agregă predicțiile. Reduce varianța specifică arborilor de decizie.
- AdaBoost construiește modele succesive care încearcă să corecteze erorile modelelor anterioare.
- La regresie, Random Forest a fost clar cel mai bun model.
- La clasificare, ensemble-urile au avut accuracy bună, dar SVM a obținut F1 mai bun datorită recall-ului mai ridicat.

Rezultate ensemble - regresie

Model	MAE	RMSE	R2
Random Forest Regressor	70,678	138,638	0.8702
AdaBoost Regressor	391,833	430,452	-0.2511

Rezultate ensemble - clasificare

Model	Acc	Prec	Recall	F1	AUC
Random Forest Classifier	0.7651	0.5520	0.5995	0.5747	0.8141
AdaBoost Classifier	0.7972	0.6537	0.4973	0.5649	0.8424

10. Salvarea modelelor și GUI-ul Streamlit

După evaluare, pipeline-ul complet al celui mai bun model este salvat cu `joblib`. Artefactul salvat conține nu doar estimatorul, ci și preprocesarea și schema de input folosită de aplicație.

- `outputs/models/house\_price\_best\_model.joblib` - modelul Random Forest Regressor.
- `outputs/models/telco\_churn\_best\_model.joblib` - modelul Support Vector Machine.
- `app.py` - aplicație Streamlit cu două taburi: House Price Regression și Telco Churn Classification.
- Formularele sunt generate din schema de feature-uri salvată în artefact, astfel încât interfața rămâne sincronizată cu modelul.

Fragment de cod - salvare artefact și GUI

```
def save_artifact(artifact: dict[str, Any], path) -> None:
    joblib.dump(artifact, path)

def render_input_form(artifact: dict, form_key: str) -> pd.DataFrame | None:
    metadata = artifact["metadata"]
    schema = metadata["feature_schema"]
    features = metadata["features"]

    with st.form(form_key):
        values = {}
        columns = st.columns(2)
        for idx, feature in enumerate(features):
            field = schema[feature]
            with columns[idx % 2]:
                if field["kind"] == "numeric":
                    value = st.number_input(
                        feature,
                        value=float(field["default"]),
                        format="%.4f",
                    )
                else:
                    choices = field.get("choices", [""])
                    default = field.get("default", choices[0])
                    default_index = choices.index(default) if default in choices else 0
                    value = st.selectbox(feature, choices, index=default_index)
            values[feature] = value

        submitted = st.form_submit_button("Predict")

    if not submitted:
        return None

    return pd.DataFrame([values], columns=features)
```

11. Limitări și direcții de îmbunătățire

- Pentru regresie, prețurile imobiliare au outliers puternici; o variantă îmbunătățită ar putea modela `log(price)`.
- Pentru churn, alegerea metricii influențează modelul final. Dacă obiectivul ar fi maximizarea profitului, ar trebui introdusă o funcție de cost business.
- SVM a fost selectat după F1, dar pentru producție ar fi utilă calibrarea probabilităților.
- GUI-ul este minimal: bun pentru demonstrație, dar nu include autentificare, logging, monitorizare drift sau bază de date.
- Random Forest este performant, dar modelul salvat pentru case este mare; pentru deploy real ar putea fi testate modele mai compacte sau optimizare de hiperparametri.

12. Concluzie

Proiectul acoperă complet fluxul studiat la laborator: date, analiză, preprocesare, modele, evaluare, ensemble-uri, salvare și interfață. Cele două module nu sunt legate prin același domeniu de date, ci prin aceeași metodologie de Machine Learning aplicată la două tipuri diferite de task-uri. Rezultatul este o aplicație demonstrativă care poate prezice atât o valoare numerică, cât și o clasă, folosind modele antrenate și salvate local.

Comenzi utile

```
..\env\Scripts\Activate.ps1  
python -m src.train  
streamlit run app.py
```